

To install Julia:

- Visit <https://julialang.org/downloads/oldreleases/> to install v1.10.2 of Julia
 - If using a Mac: Go to About this Mac (click on the Apple logo in the upper left corner)
 - If the “Chip” or “Processor” entry contains “Intel”, then download x86_64 package
 - Otherwise, if you have “Apple” chip or “M#” chip, where # is any number, then download aarch64 package
 - If using Windows: Go to Start → Settings → System → About, and look for the System type entry under Device specifications.
 - If the chip is Intel or is x86, download the x86_64 package
 - Otherwise, download the i686 package
 - If using Linux: Open the Terminal, then type the following:
 - `uname -m`
 - The output will correspond to which package you need to download.
- Once Julia is installed, you can open the Terminal application and type `julia`
 - Then press the Enter key. If this works, you should see something like the following:

```
| Documentation: https://docs.julialang.org
| Type "?" for help, "]?" for Pkg help.
| Version 1.10.5 (2024-08-27)
| Official https://julialang.org/ release
```

To install all of the necessary packages:

- Go to the website https://github.com/irishryoon/lung_ECM_TDA to and download the code (press the green “Code” button and select “Download ZIP”)
- Unzip the downloaded code and move the resulting folder to whatever directory you want.
- Open Julia using the same lines of code as before
- Type the following:
 - `cd("Path_to_the_code_directory")`, then press Enter
 - For example, if I moved the downloaded folder to `/home/myusername/Documents/downloaded_folder`, I would type `cd("~/Documents/downloaded_folder")`
 - `using Pkg`

- `Pkg.activate("." )`
- `Pkg.instantiate()`

To run a file in Julia:

1. Open Julia and install all packages as above. To run a file, type following into the terminal:

- `include("path_to_file/filename.jl")`
- For example:
 - `include("./src/compute_persistence.jl")`

Troubleshooting common errors:

1. You get an error saying something to the equivalent of “Package XXX not found” or “Package XXX is missing”.
 - This error just means that you haven’t installed that particular library on your laptop. To fix this, simply type in the following:
 - `using Pkg`
 - `Pkg.add(“XXX”)`
 - `Using XXX`
 - This should fix that issue.
2. If problem (1) happens every time you start a new Julia session:
 - This is happening because you have not activated the correct environment (i.e., you are not accessing the correct “Project.toml” and “Manifest.toml” files). The most likely reason for this is that you are not in the correct directory.
 - To fix this, find which directory you are in:
 - `pwd()`
 - Then change directory to the original folder:
 - `cd(“Path_to_the_code_directory”)`
 - Then access the correct Project.toml and Manifest.toml files:
 - `using Pkg`
 - `Pkg.activate("." )`
 - `Pkg.instantiate()`
3. If you have problems installing the PyCall library in Julia:
 - This error likely arises because you haven’t linked a version of Python to Julia so that the latter can call Python. The website <https://github.com/JuliaPy/PyCall.jl?tab=readme-ov-file> provides examples of fixes that you can do. The easiest workaround currently is to add the Conda.jl library and then install + build PyCall (it will automatically link to the Conda.jl version of Python)
 - `using Pkg`

```
▪ Pkg.add("Conda");  
▪ Pkg.add("PyCall");  
▪ Pkg.build("PyCall");  
▪ Using PyCall;
```

- Note however that this version of Python will NOT correspond to the one accessed via your terminal.
4. If you run `include("../src/TDA.jl")` and you keep getting an error saying one of the packages cannot be compiled:
- This error is because one of the packages in `TDA.jl` relies on an earlier version of Julia that uses slightly different syntax. This just means that your current version of Julia is too far upgraded. You will have to go to <https://julialang.org/downloads/oldreleases/> to install v.1.10.2 of Julia, using the instructions above.